```python
# ============================================================
# ADDITIONAL EXERCISE 1: Unstructured Data Aggregator
# Student: Bacatan | ID: TUPM-26-3609
# ============================================================

from collections import defaultdict, Counter

# --- Required Student Variables ---
SEED_NUM        = 9                 # Last digit of TUPM-26-3609
STUDENT_MAJOR   = "BSECE"           # Current degree program
FAVORITE_ARTIST = "JUST HUSH"            # Favorite local artist
LOCAL_HAZARD    = "EARTHQUAKE"      # Local emergency event
CONTROL_NUM     = max(3, SEED_NUM)  # max(3, 9) = 9

print("=== STUDENT VARIABLES ===")
print(f"SEED_NUM       : {SEED_NUM}")
print(f"STUDENT_MAJOR  : {STUDENT_MAJOR}")
print(f"FAVORITE_ARTIST: {FAVORITE_ARTIST}")
print(f"LOCAL_HAZARD   : {LOCAL_HAZARD}")
print(f"CONTROL_NUM    : {CONTROL_NUM}")
print()

# --- Base Application Stream ---
stream = [
    ("TUP Manila",   "BSECE"),
    ("TUP Taguig",   f"BSECE_v{CONTROL_NUM}"),
    ("TUP Manila",   f"BSECE_v{CONTROL_NUM}"),
    ("TUP Manila",   "BSECE"),
    ("TUP Visayas",  STUDENT_MAJOR),
    ("TUP Taguig",   "BSECE"),
    ("TUP Manila",   "BSECE"),
]

# --- Data Expansion: Add exactly (CONTROL_NUM + 3) = 12 custom tuples ---
custom_applications = [
    ("TUP Manila",   "BSECE"),          # Custom 1
    ("TUP Manila",   STUDENT_MAJOR),    # Custom 2 - BSECE
    ("TUP Taguig",   STUDENT_MAJOR),    # Custom 3 - BSECE
    ("TUP Visayas",  "BSECE"),          # Custom 4
    ("TUP Manila",   "BSECE"),          # Custom 5
```

```python
# --- Data Expansion: Add exactly (CONTROL_NUM + 3) = 12 custom tuples ---
custom_applications = [
    ("TUP Manila",   "BSECE"),          # Custom 1
    ("TUP Manila",   STUDENT_MAJOR),    # Custom 2 - BSECE
    ("TUP Taguig",   STUDENT_MAJOR),    # Custom 3 - BSECE
    ("TUP Visayas",  "BSECE"),          # Custom 4
    ("TUP Manila",   "BSECE"),          # Custom 5
    ("TUP Taguig",   "BSECE"),          # Custom 6
    ("TUP Manila",   "BSECE"),          # Custom 7
    ("TUP Visayas",  STUDENT_MAJOR),    # Custom 8 - BSECE
    ("TUP Manila",   STUDENT_MAJOR),    # Custom 9 - BSECE
    ("TUP Taguig",   "BSECE"),          # Custom 10
    ("TUP Visayas",  "BSECE"),          # Custom 11
    ("TUP Manila",   "BSECE"),          # Custom 12
]

stream.extend(custom_applications)

print(f"=== EXPANDED APPLICATION STREAM (Total: {len(stream)}) ===")
for entry in stream:
    print(f"  {entry}")
print()

# --- Algorithm Phase 1: Aggregation via defaultdict ---
campus_programs = defaultdict(list)

for campus, program in stream:
    campus_programs[campus].append(program)

print("=== PHASE 1: AGGREGATED APPLICATIONS BY CAMPUS ===")
for campus, programs in campus_programs.items():
    print(f"  {campus}: {programs}")
print()

# --- Algorithm Phase 2: Frequency Analysis via Counter ---
print("=== PHASE 2: FREQUENCY ANALYSIS PER CAMPUS ===")
campus_counters = {}

for campus, programs in campus_programs.items():
    counter = Counter(programs)
    campus_counters[campus] = counter
```

```python
for campus, programs in campus_programs.items():
    counter = Counter(programs)
    campus_counters[campus] = counter
    print(f"\n  [{campus}]")
    for program, count in counter.most_common():
        print(f"    {program}: {count} application(s)")

# --- Top Program at TUP Manila ---
manila_top = campus_counters["TUP Manila"].most_common(1)
print()
print("=== MOST DEMANDED PROGRAM: TUP MANILA ===")
print(f"  Program : {manila_top[0][0]}")
print(f"  Count   : {manila_top[0][1]} application(s)")
print()

# --- Summary Table Output ---
print("=== ASSESSMENT TABLE OUTPUT ===")
print(f"  CONTROL_NUM                        : {CONTROL_NUM}")
print(f"  Custom tuples added (CONTROL_NUM+3): {CONTROL_NUM + 3}")
print(f"  Total stream size after expand     : {len(stream)}")
print(f"  TUP Manila - Total applications    : {len(campus_programs['TUP Manila'])}")
print(f"  TUP Taguig - Total applications    : {len(campus_programs['TUP Taguig'])}")
print(f"  TUP Visayas - Total applications   : {len(campus_programs['TUP Visayas'])}")
print(f"  Manila most_common(1) result       : {manila_top}")
```

```
=== STUDENT VARIABLES ===
SEED_NUM       : 9
STUDENT_MAJOR  : BSECE
FAVORITE_ARTIST: JUST HUSH
LOCAL_HAZARD   : EARTHQUAKE
CONTROL_NUM    : 9

=== EXPANDED APPLICATION STREAM (Total: 19) ===
  ('TUP Manila', 'BSECE')
  ('TUP Taguig', 'BSECE_v9')
  ('TUP Manila', 'BSECE_v9')
  ('TUP Manila', 'BSECE')
  ('TUP Visayas', 'BSECE')
  ('TUP Taguig', 'BSECE')
  ('TUP Manila', 'BSECE')
  ('TUP Manila', 'BSECE')
  ('TUP Manila', 'BSECE')
  ('TUP Taguig', 'BSECE')
  ('TUP Visayas', 'BSECE')
  ('TUP Manila', 'BSECE')
  ('TUP Taguig', 'BSECE')
  ('TUP Manila', 'BSECE')
  ('TUP Visayas', 'BSECE')
  ('TUP Manila', 'BSECE')
  ('TUP Taguig', 'BSECE')
...
  TUP Manila - Total applications   : 10
  TUP Taguig - Total applications   : 5
  TUP Visayas - Total applications  : 4
  Manila most_common(1) result       : [('BSECE', 9)]
```

```python
# ============================================================
# ADDITIONAL EXERCISE 2: Jaccard Similarity Engine
# Student: Bacatan | ID: TUPM-26-3609
# ============================================================

# --- Required Student Variables ---
SEED_NUM        = 9
FAVORITE_ARTIST = "Just Hush"
CONTROL_NUM     = max(3, SEED_NUM)  # = 9

print("=== STUDENT VARIABLES ===")
print(f"FAVORITE_ARTIST : {FAVORITE_ARTIST}")
print(f"CONTROL_NUM     : {CONTROL_NUM}")
print()

# --- Initialize Festival Lineup Set ---
festival = {
    "Hev Abi",
    "Deny",
    "La Mave",
    "Nasty Kidd",
    FAVORITE_ARTIST,
    "Allison Shore",
    f"Hip-Hop Artist {CONTROL_NUM}"
}

# --- Initialize User Profile Sets ---
user_a = {"Hev Abi", "La Mave", "Bugoy na Koykoy", "Deny", FAVORITE_ARTIST}
user_b = {FAVORITE_ARTIST, "Uncle Dags", "Illest Morena", f"Hip-Hop Artist {CONTROL_NUM}", "Al James"}

# --- Data Expansion: Add exactly CONTROL_NUM (9) new artists to festival ---
new_artists = {
    "Gloc-9",
    "Francis M",
    "Andrew E.",
    "Loonie",
    "Abra",
    "Skusta Clee",
    "Flow G",
    "Shehyee",
```

```python
# Sanity checks: exactly CONTROL_NUM artists, none already in the festival
assert len(new_artists) == CONTROL_NUM, "new_artists must contain exactly CONTROL_NUM artists"
assert new_artists.isdisjoint(festival), "new_artists must not overlap the existing festival"

festival.update(new_artists)

print("=== EXPANDED FESTIVAL LINEUP ===")
print(f"  {sorted(festival)}")
print(f"  Total Artists in Festival: {len(festival)}")
print()

# --- Algorithm: Jaccard Similarity for User A ---
intersection_a = user_a.intersection(festival)
union_a        = user_a.union(festival)
jaccard_a      = (len(intersection_a) / len(union_a)) * 100

print("=== USER A: JACCARD SIMILARITY ===")
print(f"  User A Profile      : {sorted(user_a)}")
print(f"  Intersection (A∩F)  : {sorted(intersection_a)}  | Size: {len(intersection_a)}")
print(f"  Union (A∪F)         : Size = {len(union_a)}")
print(f"  Jaccard Similarity  : {len(intersection_a)} / {len(union_a)} x 100 = {jaccard_a:.2f}%")
print()

# --- Algorithm: Jaccard Similarity for User B ---
intersection_b = user_b.intersection(festival)
union_b        = user_b.union(festival)
jaccard_b      = (len(intersection_b) / len(union_b)) * 100

print("=== USER B: JACCARD SIMILARITY ===")
print(f"  User B Profile      : {sorted(user_b)}")
print(f"  Intersection (B∩F)  : {sorted(intersection_b)}  | Size: {len(intersection_b)}")
print(f"  Union (B∪F)         : Size = {len(union_b)}")
print(f"  Jaccard Similarity  : {len(intersection_b)} / {len(union_b)} x 100 = {jaccard_b:.2f}%")
print()

# --- Set Difference: User A's Dealbreakers ---
dealbreakers_a = user_a.difference(festival)

print("=== USER A: DEALBREAKERS ===")
```

```python
# --- Assessment Table Summary ---
print("=== ASSESSMENT TABLE OUTPUT ===")
print(f"  CONTROL_NUM Used                        : {CONTROL_NUM}")
print(f"  New artists added via .update()         : {sorted(new_artists)}")
print(f"  Expanded Festival Size                  : {len(festival)}")
print(f"  User A - Intersection Size              : {len(intersection_a)}")
print(f"  User A - Union Size                     : {len(union_a)}")
print(f"  User A - Jaccard Similarity             : {jaccard_a:.2f}%")
print(f"  User B - Intersection Size              : {len(intersection_b)}")
print(f"  User B - Union Size                     : {len(union_b)}")
print(f"  User B - Jaccard Similarity             : {jaccard_b:.2f}%")
print(f"  User A Dealbreakers                     : {sorted(dealbreakers_a)}")
```

```
=== STUDENT VARIABLES ===
FAVORITE_ARTIST : Just Hush
CONTROL_NUM     : 9

=== EXPANDED FESTIVAL LINEUP ===
  ['Abra', 'Allison Shore', 'Andrew E.', 'Deny', 'Flow G', 'Francis M', 'Gloc-9', 'Hev Abi', 'Hip-Hop Arti
  Total Artists in Festival: 16

=== USER A: JACCARD SIMILARITY ===
  User A Profile      : ['Bugoy na Koykoy', 'Deny', 'Hev Abi', 'Just Hush', 'La Mave']
  Intersection (A∩F)  : ['Deny', 'Hev Abi', 'Just Hush', 'La Mave']  | Size: 4
  Union (AUF)         : Size = 17
  Jaccard Similarity  : 4 / 17 x 100 = 23.53%

=== USER B: JACCARD SIMILARITY ===
  User B Profile      : ['Al James', 'Hip-Hop Artist 9', 'Illest Morena', 'Just Hush', 'Uncle Dags']
  Intersection (B∩F)  : ['Hip-Hop Artist 9', 'Just Hush']  | Size: 2
  Union (BUF)         : Size = 19
  Jaccard Similarity  : 2 / 19 x 100 = 10.53%

=== USER A: DEALBREAKERS ===
  Artists User A likes but NOT in festival: ['Bugoy na Koykoy']

=== ASSESSMENT TABLE OUTPUT ===
  CONTROL_NUM Used                        : 9
...
  User B - Intersection Size              : 2
  User B - Union Size                     : 19
  User B - Jaccard Similarity             : 10.53%
  User A Dealbreakers                     : ['Bugoy na Koykoy']
```

```python
# ============================================================
# ADDITIONAL EXERCISE 3: Stateful Sliding Window Filter
# Student: Bacatan | ID: TUPM-26-3609
# ============================================================

from collections import deque, Counter

# --- Required Student Variables ---
SEED_NUM        = 9
LOCAL_HAZARD    = "EARTHQUAKE"
CONTROL_NUM     = max(3, SEED_NUM)  # = 9

print("=== STUDENT VARIABLES ===")
print(f"LOCAL_HAZARD : {LOCAL_HAZARD}")
print(f"CONTROL_NUM  : {CONTROL_NUM}")
print()

# --- Initialize deque with maxlen = 5 ---
rolling_buffer = deque(maxlen=5)

# --- Base Burst of Alerts ---
burst = [
    (CONTROL_NUM,     "WEATHER"),
    (CONTROL_NUM + 1, "TRAFFIC"),
    (CONTROL_NUM + 2, "WEATHER"),
    (CONTROL_NUM + 3, "WEATHER"),
    (CONTROL_NUM + 4, LOCAL_HAZARD),
    (CONTROL_NUM + 5, "WEATHER"),
    (CONTROL_NUM + 6, "TRAFFIC"),
]

# --- Data Expansion: Append exactly (CONTROL_NUM + 2) = 11 custom alerts ---
# Timestamps continue sequentially from CONTROL_NUM + 7 onward
custom_alerts = [
    (CONTROL_NUM + 7,  LOCAL_HAZARD),
    (CONTROL_NUM + 8,  "TRAFFIC"),
    (CONTROL_NUM + 9,  LOCAL_HAZARD),
    (CONTROL_NUM + 10, "WEATHER"),
    (CONTROL_NUM + 11, "TRAFFIC"),
    (CONTROL_NUM + 12, "WEATHER"),
```

```python
# --- Data Expansion: Append exactly (CONTROL_NUM + 2) = 11 custom alerts ---
# Timestamps continue sequentially from CONTROL_NUM + 7 onward
custom_alerts = [
    (CONTROL_NUM + 7,  LOCAL_HAZARD),
    (CONTROL_NUM + 8,  "TRAFFIC"),
    (CONTROL_NUM + 9,  LOCAL_HAZARD),
    (CONTROL_NUM + 10, "WEATHER"),
    (CONTROL_NUM + 11, "TRAFFIC"),
    (CONTROL_NUM + 12, "WEATHER"),
    (CONTROL_NUM + 13, LOCAL_HAZARD),
    (CONTROL_NUM + 14, "TRAFFIC"),
    (CONTROL_NUM + 15, "WEATHER"),
    (CONTROL_NUM + 16, LOCAL_HAZARD),
    (CONTROL_NUM + 17, "TRAFFIC"),
]
burst.extend(custom_alerts)

print(f"=== EXPANDED BURST LIST (Total: {len(burst)} alerts) ===")
for alert in burst:
    print(f"  {alert}")
print()

# --- Algorithm: Sliding Window Filter ---
print("=== SLIDING WINDOW PROCESSING ===")
rejected_count = 0
accepted_count = 0

for alert in burst:
    timestamp, category = alert

    # Extract current categories from deque using list comprehension
    current_categories = [item[1] for item in rolling_buffer]

    # Count category frequencies in current window
    freq_counter = Counter(current_categories)

    # Spam Rule: reject if category appears 2 or more times already
    if freq_counter[category] >= 2:
        print(f"  [REJECTED] Timestamp {timestamp} | Category: {category} "
              f"(already {freq_counter[category]}x in buffer)")
```

```
        rejected_count += 1
    else:
        rolling_buffer.append(alert)
        print(f"  [ACCEPTED] Timestamp {timestamp} | Category: {category} "
              f"| Buffer size: {len(rolling_buffer)}")
        accepted_count += 1

print()

# --- Final State of Deque ---
print("=== FINAL DEQUE STATE ===")
for item in rolling_buffer:
    print(f"  {item}")
print()

# --- Sum of Timestamps in Final Deque ---
timestamp_sum = sum(item[0] for item in rolling_buffer)

print("=== ASSESSMENT TABLE OUTPUT ===")
print(f"  CONTROL_NUM Used                          : {CONTROL_NUM}")
print(f"  LOCAL_HAZARD Used                         : {LOCAL_HAZARD}")
print(f"  Custom alerts added (CONTROL_NUM + 2)     : {CONTROL_NUM + 2}")
print(f"  Total alerts in expanded burst            : {len(burst)}")
print(f"  Total Accepted alerts                     : {accepted_count}")
print(f"  Total Rejected alerts                     : {rejected_count}")
print(f"  Final Deque Contents                      : {list(rolling_buffer)}")
print(f"  Sum of Timestamps in Final Deque          : {timestamp_sum}")
```

```
=== STUDENT VARIABLES ===
LOCAL_HAZARD : EARTHQUAKE
CONTROL_NUM  : 9

=== EXPANDED BURST LIST (Total: 18 alerts) ===
  (9, 'WEATHER')
  (10, 'TRAFFIC')
  (11, 'WEATHER')
  (12, 'WEATHER')
  (13, 'EARTHQUAKE')
  (14, 'WEATHER')
  (15, 'TRAFFIC')
  (16, 'EARTHQUAKE')
  (17, 'TRAFFIC')
  (18, 'EARTHQUAKE')
  (19, 'WEATHER')
  (20, 'TRAFFIC')
  (21, 'WEATHER')
  (22, 'EARTHQUAKE')
  (23, 'TRAFFIC')
  (24, 'WEATHER')
  (25, 'EARTHQUAKE')
  (26, 'TRAFFIC')

=== SLIDING WINDOW PROCESSING ===
...
  Total Accepted alerts                : 12
  Total Rejected alerts                : 6
  Final Deque Contents                 : [(20, 'TRAFFIC'), (21, 'WEATHER'), (22, 'EARTHQUAKE'), (23, 'TRAFFIC'), (25, 'EARTHQUAKE')]
  Sum of Timestamps in Final Deque     : 111
```